On-line Construction of Suffix Trees

1st Eren Darak
Computer Science (of Aff.)
Özyeğin University (of Aff.)
İstanbul, Turkey
eren.darak@ozu.edu.tr

2st Ahmet Berkay Arslanpençe
Computer Science (of Aff.)
Özyeğin University (of Aff.)
İstanbul, Turkey
berkay.arslanpence@ozu.edu.tr

***Abstract*—An on-line algorithm is presented for constructing the suffix tree for a given string in time linear in the length of the string. The new algorithm has the desirable property of processing the string symbol by symbol from left to right. It always has the suffix tree for the scanned part of the string ready. The method is developed as a linear-time version of a very simple algorithm for (quadratic size) suffix tries. Regardless of its quadratic worst case this latter algorithm can be a good practical method when the string is not too long. Another variation of this method is shown to give, in a natural way, the well-known algorithms for constructing suffix automata (DAWGs).**

I. Introduction

A suffix tree is a trie-like data structure that holds all suffixes of a text as keys and poisitons as values. Suffix trees fastens up the process of many string operations. However linear time algorithms in the literature are relatively difficult to understand. In this paper, development of an alternative algorithm for suffix trees will be discussed. This new alternative algorithm has a significant property of being on-line meaning processing the string from left to right starting from longest suffix and traverses to smaller suffixes which is opposite of the Weiner[13] method that processes the string from right to left in increasing order starting from the shortest suffix. The new algorithm, processes the string one symbol at a time and keeps the suffix tree up to date with the string that has been processed up until that point. Our algorithm has some similarities with McCreight[9] in terms of adding suffixes in decreasing order of their length.

II. Constructing Suffix Tries

A. Maintaining the Integrity of the Specifications

Let B any arbitrary string with (s:t) where 's' is the starting suffix, and the 't' is the ending suffix of B. For each B; all combinations of an ordered 's', 't' and "x" where B="sxt" are the sub-strings of B. Those all subsets are denoted F(B). A suffix trie is a representation of F(B) in a tree-based data structure.

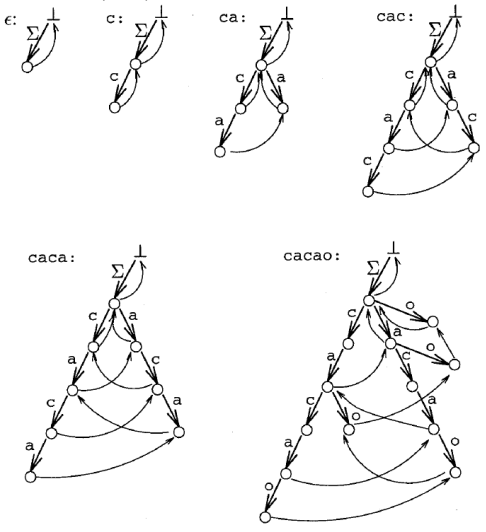
In the first manner, SuffixTrie(B) is defined as an empty trie. The initialization starts with assigning the root that is an empty node. After the initialization of the SuffixTrie(B), each coming element after 's' will be added to the trie both individually and to the existing leaf nodes as a second element. If B="skt", then after the first two steps, SuffixTrie(B) has two nodes which are an empty node and its child which is 's'. After the first three steps SuffixTrie(B) four nodes which are an empty node and its children both 'k' and 's'. And 's' has a child node which is 'k'.

Another propert of a suffix trie in this case is suffix links. Suffix links allow traversal between nodes. When a new node is created, a return link is added to the existing node behind it. In addition, if a new leaf is created and this leaf has a lower depth than the current leaf, a link edge is added between it and that node.

Algorithm 1

$r \leftarrow top$;
while $g(r, t_i)$ is undefined **do**
  create new state r' and new transition $g(r, t_i) = r'$;
  if $r \neq top$ **then** create new suffix link $f(oldr') = r'$;
  $oldr' \leftarrow r'$;
  $r \leftarrow f(r)$;
create new suffix link $f(oldr') = g(r, t_i)$;
$top \leftarrow g(top, t_i)$.

According to the Algorithm 1, the running time becomes quadratic due to the string iteration. Since every element will be iterated one time to every node, it will run O(N) time. Nevertheless, the algorithm will run the iteration at the size of B, which costs O(N) time as well. This makes run time of the algorithm O(n*n).



III. Sufix Trees

As briefly discussed in the Introduction, a suffix tree or PAT tree is a tree algorithm derived from tries that we discussed in

section 2. In a suffix tree, a string S with length n has exactly m leaves each representing a suffix in S. Each path in a suffix tree starting from the root and going through the leaf nodes, will construct suffixes of S.

A. States

Suffix trees include some states. One of them is explicit states, which consist of branching states with at least 2 outgoing edges or leaves with no outgoing edges. Notice that the root node is always an explicit state since it always has at least 2 outgoing edges. Another type of state is implicit states with exactly one transition but not explicitly represented.

B. Transition in a Suffix Tree

Transitions between explicit states are stored with a pair of pointers which are the start and end positions of the substring and with this implementation storage usage is being reduced. There are also links providing efficient construction by connecting explicit states in a suffix tree called ”suffix links”.

C. Final States

Final states can be added when required, generally by assigning a unique symbol to T which ensures all suffixes terminate at leaves. This approach guarantees a one-to-one relation between the leaves of the suffix tree and the suffixes of T.

D. Space and Construction Complexity

Suffix trees are O(n) size algorithms where n is the length of the String S and the algorithm can also be constructed in O(n) time by using methods such as Ukkonen’s algorithm which will be discussed in the next section.

IV. On-Line Construction of Suffix Trees

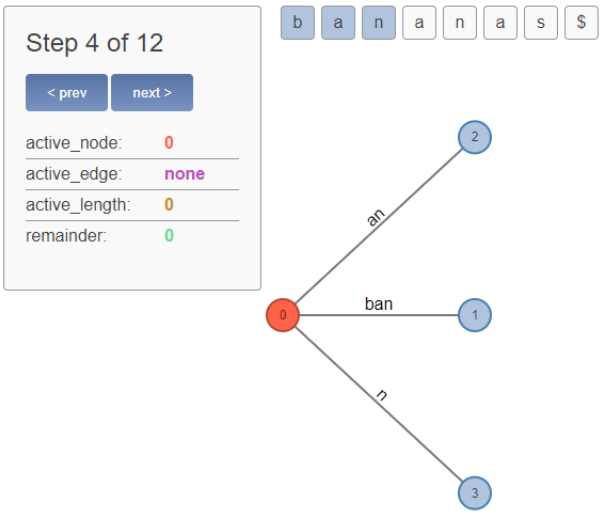
Let B = “sxt” where s is the starting suffix, t is the ending suffix and with “x” the length of the B string is m. The algorithm divided into m steps. One step for each character in the string. Initialization starts with adding the empty root node. It is same as initializing suffix tries. But the difference is that B should have a unique character to end the algorithm correctly. Due to that, in the initialization the algorithm adds a unique character to the string. From now on, B = “sxt$”. Furthermore, the edges added to the tree will be represented by pointers to decrease the space complexity. That is, the first edge will be represented as [1, m+1]. The second one will be [2, m+1] and goes same for the other edges.

There is also a way to decrease the time complexity of traversal when the algorithm goes to a specific node. This way is called suffix links. A suffix link exists for an internal node v with a path-label xS (where ‘x’ is a single character and S is the continuation of that substring) if there is another node B(v) with path-label S. It creates an edge to traverse easily from v to B(v). The root node itself does not have a suffix link since it is not considered as an internal node.

After the initialization step, there will be addition steps of each character in the string. Those additions are handled according to their conditions. For each condition, there are some special steps that occur.

The case of non-repeated substring: In this condition, the algorithm adds each character in a same way. First, the step j with character ‘a’ will be added to all non-repeated substring’s leaf nodes. If some nodes have any children, it will just be added to their children’s nodes. That is, every leaf node will have ‘a’ as their child node at the end. Additionally, a leaf node from the root node will be added to the tree, which is a single ‘a’ node. Due to those steps, there will be no edges from any node that starts with the same character. If the algorithm does not encounter any repeated substring or character, it will add the unique character to the tree in the same way and finish itself. That will take O(m) time for the construction.

Construct tree T1 For i from 1 to m-1 do begin phase i+1 For j from 1 to i+1 begin extension j Find the end of the path from the root labelled S[j..i] in the current tree. Extend that path by adding character S[i+l] if it is not there already end; end;

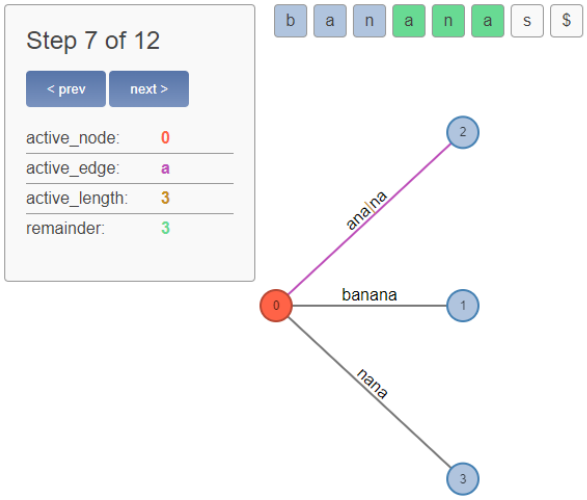


The case of repeated substring: This is the condition where things will change according to some special tricks. Suppose the algorithm tries to add a character ‘g’ where at least one path starts with that character. In this case, there are two important implementations that the algorithm should cover. The first one is the remainder. Remainder is an integer which holds the number of suffixes that algorithms should insert at that step. The remainder should always be incremented by one when a new step arises. And it should always be decreased by one when that step is completed correctly. The second important implementation is active point. It consists of three things: active node, active edge, active length. Active node is the node that the algorithm interests. It is the root node at default. Active edge is the edge leaving active node. It is null at default. Active length is the depth of the node that algorithm interests from the active node. It is zero at default since the algorithm starts with the active point at the beginning. Those implementations have always been present in the code without considering conditions. It will be represented as ActivePoint(active_node, active_edge, active_length).

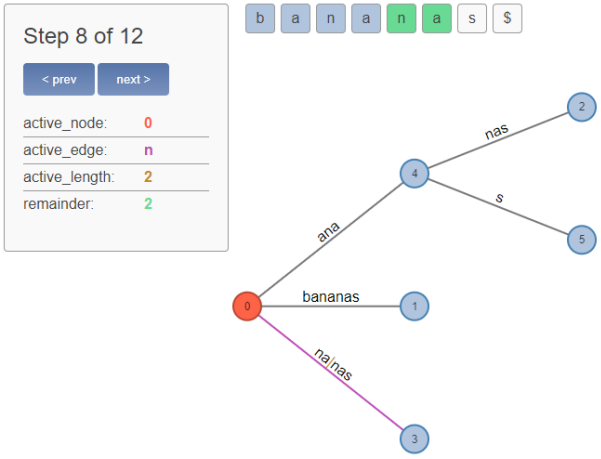
At the point when ‘g’ will be added to the tree, those changes will happen: First the remainder will be increased by one. After that, as in the non-repeating condition, the suffix ‘g’ will be added to all the leaf nodes. But the difference appears from not adding an external edge of ‘g’ itself to the root node. This means that there is no successful insertion of the new suffix ‘g’. At this point, the remainder does not decrease. The active point adjusted accordingly. Since ‘g’ is the first suffix that is found repeating, active node will remain the same as root node. Active edge will be changed to the edge starts with ‘g’ since the algorithm’s interest is now the edge starting with ‘g’ and its leaf nodes. Active length will be one. ActivePoint(root, ‘g’, 1).

This iteration continues repeatedly to find a different suffix. If B does not have any different suffix after that point, it will eventually occur the unique suffix added to the end at the initialization part, which is ‘$’.

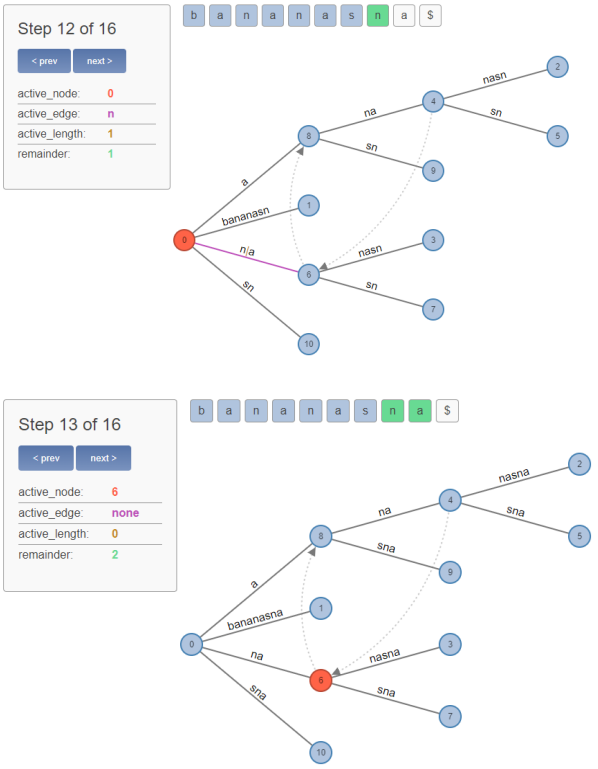
Let ‘h’ be the suffix where the algorithm finds the non-repeated character. And let ‘g’ be the substring that repeats at that edge before ‘h’ occurs. At this point, ActivePoint(root, ’g’, 1). When the algorithm finds out that the next letter is not ‘h’, it will create an internal node that splits the active point. And that internal node will have two outgoing edges. The first edge will have the remaining characters after the active point and the current character ‘g’. The leaf node represents the edge with ‘g’ will have the value of (step_number – active_length). Assume B=”bananas$”, “g” is “ana” (repeating substring) and ‘h’ is ‘s’. The split occurs as “ana—nas” -¿ “ana”, “nas”, ‘s’ where “ana” is the internal node said above, and “nas” and ‘s’ are the leaf nodes of that internal node.



When a new split occurs, the first thing the algorithm does is to update the active point. Active node remains the same as the step we encounter has root node previously as active node. Active edge becomes the first suffix of the new substring that will be inserted. For the above example, it will be ‘n’ since “nas” is the substring that will be inserted after “anas”. And active length will be decreased by one. That is, ActivePoint(root, ‘n”, 2). Additionally, since the last iteration (“anas”) added successfully to the tree, the remainder will be decreased by one and now the remainder consists of (“nas”, “as”, “s”).



As the algorithm stays in the same step, it will apply the same approach for the elements of the remainder. It will create a new internal node for “nas” which is “na” and add its children “nas” and ‘s’. It will also decrease the remainder to two where remainder is (“as”, “s”). And ActivePoint(root, ‘a’, 1). When the algorithm adds the internal nodes one by one due to splits, this is the place where the suffix links will be added to decrease traversal time complexity. The algorithm adds a suffix link from the first internal node (in our case it is “ana”) to its child internal nodes (in our case it is “na”). After that, it continues to the same way until the remainder reaches zero. Same applies for the case “as”. It will split it and create an internal node as “a”. It will have its children “na” and “s”. It will also add the suffix link due to the creation of internal node after the first internal one. Now the active point is ActivePoint(root, ‘s’, 0) and remainder is 1 which is (“s”). The same approach applies to the last remainder. Since it was the last remainder and the step completed successfully, there will also be an additional edge connected to the root and have a value of ‘s’. For the string B now is the unique element’s turn. Adding it will not cause any problem since it is different from all possible substrings. Nevertheless, let B = “bananasna$” and after the iteration step of ‘s’, there will be a suffix that repeats itself. That is, in the “bananas” case, there is a node which has a value of “na” and now, there is a step that tries to iterate “na” again. This is where a special case occurs. The special case occurs when an exact same node tries to repeat itself. The reason why this is a special case is that if another suffix arrives, there is no chance to check it. Because there is no suffix remaining at that node. If the algorithm runs into this type of a special case, it will change the active point and move from there. The biggest change here is the change of active node. From this time forth, root active node (which is default) becomes the internal node that repeats itself. And the active edge becomes null since active node reset. It also applies for the edge length which will be 0.



The remaining part handled the same way. Scan of the appearing remainders and place them in the necessary leaf nodes and perform the split event when necessary. One thing to pay attention is that since the active node changed accordingly, the new remainders are added only to the leaves under the active node.

```
// Ukkonen's Algorithm
i = 0;
remainder = 0;
(active_node = root, active_edge = null, active_length = 0) //active point

create root node;
while S not empty{
    i++;
    remainder++;

    // S[i] is next character after active point
    if S[i] == next char after active point{
        if active_length == 0{
            active_edge = edge beginning with S[i];
        }
        active_length++;

        if active point is at the end of an edge{
            active_node = node edge is pointing to;
            active_edge = null;
            active_length = 0;
        }
    }
    // S[i] is not next character after active point
    else{
        while remainder > 0{
            if active_length == 0{
                create edge for S[i];
                create leaf node with integer value of i;
                point new edge to new leaf node;
            }
            else{
                //split edge
                create internal node and point active_edge to it;
                active_edge stops at active point; //e.g., abc|def -> abc|
                create new edge with remains of active_edge; //e.g., def
                point new edge to node that active_edge previously pointed to

                create new edge leaving new node with integer value of i for
                start and i for end; //e.g., if i= 4, [4,i]

                create new leaf node with integer value of i - active_length;
                point new edge to this leaf node;

                if not the first split edge{
                    create suffix link from previous node to current;
                }
            }

            remainder--;

            if active_node == root{
                active_edge = first letter of next remaining suffix or null; // if
                remainder == 0
                active_length-- or 0;
            }
            else{
                if node has suffix link{
                    active_node = node pointed to by suffix link;
                }
                else{
                    active_node = root;
                }
            }
        }
    }
}
```

References

[1] Brenden. (n.d.). Ukkonen's suffix tree construction animation. Retrieved May 31, 2024, from https://brenden.github.io/ukkonen-animation/

[2] GeeksforGeeks. (n.d.). Ukkonen's suffix tree construction - Part 1. GeeksforGeeks. Retrieved May 31, 2024, from https://www.geeksforgeeks.org/ukkonens-suffix-tree-construction-part-1/

[3] GeeksforGeeks. (n.d.). Ukkonen's suffix tree construction - Part 2. GeeksforGeeks. Retrieved May 31, 2024, from https://www.geeksforgeeks.org/ukkonens-suffix-tree-construction-part-2/

[4] GeeksforGeeks. (n.d.). Ukkonen's suffix tree construction - Part 3. GeeksforGeeks. Retrieved May 31, 2024, from https://www.geeksforgeeks.org/ukkonens-suffix-tree-construction-part-3/?ref=lbp

[5] GeeksforGeeks. (n.d.). Ukkonen's suffix tree construction - Part 4. GeeksforGeeks. Retrieved May 31, 2024, from https://www.geeksforgeeks.org/ukkonens-suffix-tree-construction-part-4/?ref=lbp

[6] GeeksforGeeks. (n.d.). Ukkonen's suffix tree construction - Part 5. GeeksforGeeks. Retrieved May 31, 2024, from https://www.geeksforgeeks.org/ukkonens-suffix-tree-construction-part-5/?ref=lbp

[7] GeeksforGeeks. (n.d.). Ukkonen's suffix tree construction - Part 6. GeeksforGeeks. Retrieved May 31, 2024, from https://www.geeksforgeeks.org/ukkonens-suffix-tree-construction-part-6/?ref=lbp

[8] [Ben Langmead]. (2022, July 22). Tries [Video]. Youtube. https://rb.gy/xzgq85

[9] [Ben Langmead]. (2022, July 29). Suffix trees: Basic queries [Video]. Youtube. https://www.youtube.com/watch?v=A3p6HKfz4Cs

[10] [Tarvalas]. (2021, April 20). Ukkonen's Algorithm [Video]. Youtube. https://www.youtube.com/watch?v=ALEV0Hc5dDk

[11] Ukkonen, E. (1995). On-line construction of suffix trees. Algorithmica, 14(3), 249–260. https://doi.org/10.1007/bf01206331